Shor's Algorithm in Quantum Computing

Sarbasish Chowdhury (252IS033)
Md Timsal Alam (252IS021)

Department of CSE
NITK Surathkal
CS826 – Quantum Cryptography

April 17, 2026

Outline

1. Introduction
2. Shor's Algorithm
3. QFT + QPE
4. Worked Examples
5. Conclusion

Integer Factorization Problem

- Given a large composite number:

$$N = p \times q$$

  where p and q are prime numbers.
- Goal: Determine the prime factors p and q.
- Example:

$$15 = 3 \times 5$$

- Computational Challenge:
  - Easy to multiply primes
  - Extremely hard to factor large numbers
- Classical complexity:
  - Best algorithms are **sub-exponential**
  - Not feasible for very large N

Basis of RSA Cryptosystem

- RSA is a widely used public-key cryptographic system.
- Key idea:
  - Choose two large primes p and q
  - Compute $N = p \times q$
  - Public key: (N, e)
  - Private key: d
- Security is based on:
  - Difficulty of factoring N into p and q
- Without factorization:
  - Private key cannot be efficiently derived

Impact on Cryptography

- Modern security systems rely on:
  - RSA
  - Diffie-Hellman
  - Digital signatures
- These depend on:
  - Hard mathematical problems (factorization, discrete log)
- If factorization becomes easy:
  - Encryption can be broken
  - Secure communication becomes vulnerable
- Leads to:
  - Need for **Post-Quantum Cryptography**

Introduction to Shor's Algorithm

- Proposed by **Peter Shor (1994)**.
- First quantum algorithm to efficiently solve:
  - Integer factorization
- Key idea:
  - Convert factoring into a **period finding problem**
- Uses:
  - Quantum Fourier Transform (QFT)
  - Quantum Phase Estimation
- Advantage:
  - Runs in **polynomial time**
  - Classical algorithms are sub-exponential
- Breaks RSA security assumption

Shor's Algorithm: Overview

- Goal: Factor a composite number $N = p \times q$
- Key idea:
  - Reduce factoring to **period finding**
- Define function:

$$f(x) = a^x \bmod N$$

- Find period r such that:

$$a^r \equiv 1 \pmod{N}$$

- Quantum advantage:
  - Period finding done efficiently using QFT

Step-by-Step Procedure

1. Choose a random integer a, where $1 < a < N$
2. Compute:
$$\gcd(a, N)$$
   - If $\gcd(a, N) \neq 1$, we already found a factor
3. Find the period r of:
$$f(x) = a^x \mod N$$
4. Check if r is even
5. Compute:
$$a^{r/2} \pm 1$$
6. Final factors:
$$p = \gcd(a^{r/2} - 1, N), \quad q = \gcd(a^{r/2} + 1, N)$$

Why Period Finding? (Intuition)

- Consider function:

$$f(x) = a^x \mod N$$

- This function is **periodic**
- If we find period r, we can compute:

$$\gcd(a^{r/2} \pm 1, N)$$

- Classical computers:
  - Try many values $\Rightarrow$ exponential time
- Quantum computers:
  - Use **superposition + interference**
  - Extract period efficiently using QFT

Period Finding Problem

- Given function:

$$f(x) = a^x \bmod N$$

- Goal:
  - Find smallest r such that:

$$f(x + r) = f(x)$$

- Equivalent to:

$$a^r \equiv 1 \pmod{N}$$

- Hard problem classically
- Efficient using quantum algorithms

Period of Modular Exponentiation

- Consider the function:

$$f(x) = a^x \bmod N$$

- This function is **periodic** if $\gcd(a, N) = 1$
- The **period (order)** r is the smallest positive integer such that:

$$a^r \equiv 1 \pmod{N}$$

- Example:

$$2^x \bmod 7 = 1, 2, 4, 1, 2, 4, \cdots \Rightarrow r = 3$$

- Key property:

$$a^{x+r} \equiv a^x \pmod{N}$$

- Importance in Shor's Algorithm:
  - Finding r allows factorization of N
  - Period finding is done efficiently using QFT

Mathematical Foundation

- From period definition:

$$a^r \equiv 1 \pmod{N}$$

- Rearranging:

$$a^r - 1 \equiv 0 \pmod{N}$$

- Factorization:

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1)$$

- Therefore:

$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$$

- Implies:
  - At least one term shares a factor with N

Extracting Factors

- Since:

$$N \mid (a^{r/2} - 1)(a^{r/2} + 1)$$

- Compute:

$$\gcd(a^{r/2} - 1, N)$$
$$\gcd(a^{r/2} + 1, N)$$

- These yield non-trivial factors:

$$p \text{ and } q$$

- Based on number theory:
  - If $a^{r/2} \not\equiv \pm 1 \pmod{N}$, factors are guaranteed

Important Conditions

- Condition 1:

$$\gcd(a, N) = 1$$

- Condition 2:

$$r \text{ must be even}$$

- Condition 3:

$$a^{r/2} \not\equiv -1 \pmod{N}$$

- Why?
  - If $a^{r/2} \equiv -1$, then:

$$\gcd(a^{r/2} + 1, N) = N$$

  - No non-trivial factor obtained
- If conditions fail:
  - Choose another value of a

Quantum Fourier Transform (QFT)

- Quantum analogue of the Discrete Fourier Transform (DFT)
- Operates on quantum states:

$$|x\rangle \to \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k/N} |k\rangle$$

- Transforms amplitudes into frequency domain
- Key property:
  - Extracts **periodicity** in quantum states
- Central component of Shor's Algorithm
- Advantage:
  - Classical FFT: $O(N \log N)$
  - QFT: $O((\log N)^2)$ gates

QFT Circuit Structure

- Built using:
  - Hadamard gates
  - Controlled phase rotation gates (R_k)
- Steps:
  1. Apply Hadamard to first qubit
  2. Apply controlled rotations
  3. Repeat for all qubits
  4. Reverse qubit order (SWAP gates)
- Total complexity:

$$O(n^2) = O((\log N)^2)$$

Quantum Phase Estimation (QPE)

- Goal: Estimate phase θ of eigenvalue
- Given:

$$U|v\rangle = e^{2\pi i\theta}|v\rangle$$

- Where:
  - U is a unitary operator
  - $|v\rangle$ is an eigenvector
  - $\theta \in [0, 1)$ is the phase
- Output:
  - Binary approximation of θ

Quantum Period Finding Procedure

- Step 1: Initialize registers

$$|0\rangle^{\otimes n}|1\rangle$$

- Step 2: Apply Hadamard

$$\frac{1}{\sqrt{Q}}\sum_{x}|x\rangle|1\rangle$$

- Step 3: Apply modular exponentiation

$$\frac{1}{\sqrt{Q}}\sum_{x}|x\rangle|f(x)\rangle$$

- Creates periodic structure in superposition

Connection to Shor's Algorithm

- In Shor's Algorithm:

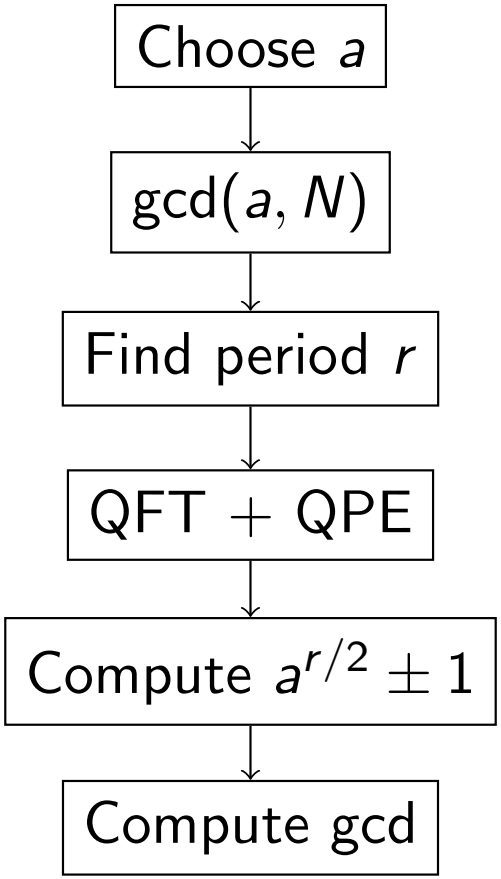
$$\theta = \frac{s}{r}$$

- QPE estimates:

$$\frac{s}{r}$$

- Using continued fractions:
  - Extract period r
- This enables:
  - Efficient factorization
- QPE + QFT = Core of Shor's speedup

Shor's Algorithm Flow



Example 1: Factor $N = 35$

Problem: Factor $N = 35$ using Shor's Algorithm

- Step (a): Choose a such that $\gcd(a, 35) = 1$
- Let $a = 2$

$$\gcd(2, 35) = 1 \quad \checkmark$$

- Step (b): Find period r of:

$$f(x) = 2^x \bmod 35$$

Example 1: Finding Period

- Compute powers:

$$2^1 = 2 \mod 35$$

$$2^2 = 4 \mod 35$$

$$2^3 = 8 \mod 35$$

$$2^4 = 16 \mod 35$$

$$2^5 = 32 \mod 35$$

$$2^6 = 64 \equiv 29 \mod 35$$

$$2^{12} \equiv 1 \mod 35$$

- Therefore, period:

$$r = 12$$

Example 1: Factor Extraction

- Step (c): Compute

$$p = \gcd(2^6 - 1, 35), \quad q = \gcd(2^6 + 1, 35)$$

- Calculate:

$$2^6 = 64$$

$$p = \gcd(63, 35) = 7$$

$$q = \gcd(65, 35) = 5$$

- Final Answer:

$$35 = 5 \times 7$$

Example 2: Factor $N = 209$

Given: $a = 22$

- Step (a): Check $\gcd(a, N)$

$$\gcd(22, 209)$$

- Factor:

$$209 = 11 \times 19$$

$$\gcd(22, 209) = 11 \neq 1$$

- Therefore:

$$p = 11, \quad q = \frac{209}{11} = 19$$

- Factorization obtained directly

Example 2 (Alternate a): Continue Shor's Algorithm

- Choose $a = 2$

$$\gcd(2, 209) = 1 \quad \checkmark$$

- Find period of:

$$f(x) = 2^x \mod 209$$

- Using computation:

$$2^{18} \equiv 1 \mod 209 \Rightarrow r = 18$$

Example 2: Factor Extraction

- Compute:

$$p = \gcd(2^9 - 1, 209), \quad q = \gcd(2^9 + 1, 209)$$

- Calculate:

$$2^9 = 512$$

$$p = \gcd(511, 209) = 19$$

$$q = \gcd(513, 209) = 11$$

- Final Answer:

$$209 = 11 \times 19$$

Time Complexity of Shor's Algorithm

- Let N be an n-bit integer
- Classical complexity (best known):
  - Number Field Sieve: sub-exponential

$$\sim e^{O(n^{1/3})}$$

- Quantum complexity:
  - Modular exponentiation: $O(n^3)$
  - Quantum Fourier Transform: $O(n^2)$
  - Continued fractions: $O(n^3)$
- Overall complexity:

$$O(n^3)$$

- Key Result:
  - **Polynomial time vs sub-exponential (classical)**

Conclusion

- Shor's Algorithm efficiently factors large integers
- Reduces factorization to:
  - Period finding problem
- Uses key quantum techniques:
  - Superposition
  - Quantum Fourier Transform (QFT)
  - Phase Estimation
- Achieves exponential speedup over classical methods
- Major implications:
  - Breaks RSA cryptography
  - Impacts global cybersecurity
- Future direction:
  - Post-quantum cryptography

Thank You